CLINIX – HOSPITAL MANAGEMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

BHAVADHARANI B

Register No.: **810023205088**



DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING – (BIT CAMPUS)

ANNA UNIVERSITY , TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Clinix-Hospital Management System "** is the Bonafide work of **BHAVADHARANI B (810023205088)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

Clinix is a full-stack Hospital Management System developed to modernize and simplify the daily operations of healthcare facilities. The system provides a centralized digital platform where patients can register, browse specialized doctors, book appointments, and view their medical history. Simultaneously, it empowers doctors to manage their daily schedules, accept or reject appointments, and mark consultations as completed. Administrators are provided with a robust control panel to manage doctors, view patient registries, and oversee all appointments system-wide.

The project is implemented using PHP for the backend, which provides a lightweight, highly efficient, and server-side scripting architecture for handling business logic and dynamic content generation. The frontend is designed using HTML, CSS, JavaScript, and Bootstrap 5 to deliver a responsive, mobile-first, and user-friendly interface. Furthermore, the system is encapsulated into a native Android application using Java and WebView, ensuring cross-platform accessibility without the need for a separate mobile codebase.

One of the main objectives of this system is to reduce manual register-keeping, minimize scheduling conflicts, and improve the overall quality of healthcare delivery. By digitizing key processes such as appointment booking, triage management, and doctor-patient interactions, the system enhances efficiency and contributes to a better clinical experience

Overall, Clinix demonstrates the practical application of full-stack development principles and modern software engineering practices. It highlights the integration of a powerful PHP-based backend with an interactive web interface and a mobile wrapper to create a complete solution for managing hospital services effectively. The project serves as a useful example of how technology can be applied to improve business operations in the healthcare industry.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1 - 8
2	COMPANY PROFILE	8 - 11
3	DAILY WORK REPORT	11-14
4	BLOCK DIAGRAM	14
5	CLINIX-HOSPITAL MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	15-17
6	PROPOSED METHOD 6.1 USER AUTHENTICATION 6.2 DOCTOR MANAGEMENT 6.3 APPOINTMENT BOOKING	27-18
7	ALGORITHM 7.1 USER LOGIN 7.2 USER REGISTRATION 7.3 APPOINTMENT BOOKING 7.4 DOCTOR TRIAGE 7.5ADMIN OPERATIOND	18-20
8	CODE IMPLEMENTATION	20-25
9	SAMPLE OUTPUT	25-29
10	ANALYSIS	29-30
11	CONCLUSION	30-31
12	FUTURE WORK	31-32
13	REFERENCES	32

1.INTRODUCTION

Clinix is a PHP Full Stack web application developed to automate and simplify the day-to-day operations of a hospital. In today's fast-growing healthcare industry, patients expect quick and reliable online services for searching for doctors, booking appointments, and receiving confirmations. Traditional hospitals that depend on manual register books, phone calls, and paperwork often face problems such as double-booking of appointment slots, delayed responses, data redundancy, and difficulty in managing patient histories. To overcome these challenges, a centralized and web-based Hospital Management System has been developed.

The primary purpose of this project is to provide a secure and user-friendly platform where patients can easily register, browse doctors based on specialization, book appointments, and cancel them if necessary. The system provides a dedicated Doctor portal to accept, reject, or complete appointment requests. It also provides an Administrator panel that enables administrators to add or remove doctors, manage patient records, and monitor all appointments efficiently. This automation reduces manual work, improves data accuracy, enhances patient satisfaction, and increases the overall productivity of the hospital.

The healthcare industry is currently undergoing a massive digital transformation, with an increasing emphasis on patient-centric care and accessibility. Patients today expect healthcare services to be as accessible and convenient as the other online services they use daily. A system like Clinix directly addresses this paradigm shift by removing geographical and temporal barriers. It ensures that critical healthcare touchpoints—such as finding the right specialist or securing a timely appointment—are available 24/7. By shifting from a reactive, paper-based model to a proactive, digital model, hospitals can significantly reduce patient wait times, optimize doctor schedules, and allocate resources more effectively, ultimately leading to a higher standard of care.

Overall, Clinix is a practical implementation of PHP Full Stack Development that demonstrates the integration of frontend technologies, PHP backend services, MySQL database management, and Android mobile integration. The project provides valuable practical knowledge in web application development, database connectivity, session management, and software engineering principles.

A standout architectural decision in the development of Clinix is the utilization of an Android WebView wrapper to deliver the mobile experience. In traditional workflows, developing a separate native mobile application requires duplicating business logic and maintaining two entirely separate codebases, which exponentially increases development time and maintenance costs. By wrapping the responsive PHP web application inside a native Android shell, Clinix achieves 100% feature parity instantly across web and mobile platforms. This approach, widely used by modern digital agencies like Exposure India, demonstrates a deep understanding of resource optimization. It highlights the ability to leverage existing server-side logic to create a seamless, app-like experience on mobile devices—complete with session persistence and hardware back-button integration—without the overhead of building a dedicated mobile backend.

The application is developed using PHP Full Stack technologies. The frontend is designed using HTML5, CSS3, JavaScript, and Bootstrap 5, which provide an attractive, responsive, and interactive user interface optimized for mobile browsers. The backend is developed using PHP 8, which handles business logic, session management, role-based access control, and communication between the frontend and the database. MySQL is used as the relational database management system to store patient details, doctor profiles, and appointment records. Additionally, an Android application is developed using Java and WebView to wrap the web application into a native mobile experience.

The system follows a modular architecture, separating concerns through distinct directories for Patient, Doctor, and Admin functionalities. The frontend communicates with the backend through form submissions and session variables, allowing efficient data exchange. This layered architecture improves maintainability and security.

Clinix consists of several important modules, including User Registration and Login, Doctor Discovery, Appointment Lifecycle Management, Profile Management, and Admin Control. Each module performs specific tasks and works together to provide a complete hospital management solution. The system includes role-based access, allowing patients, doctors, and admins to access only the features relevant to their roles.

1.1 OBJECTIVES OF THE INTERNSHIP

specific objectives:

The main objective of Clinix is to develop a secure and user-friendly web application using PHP Full Stack technologies. The system is designed to simplify hospital operations by providing an online platform for patients, doctors, and administrators.

The objectives of the project are:

- To develop a web-based application using PHP, HTML, CSS, JavaScript, Bootstrap, and MySQL.
- To provide secure user registration and login with Bcrypt password hashing.
- To enable patients to view doctors filtered by medical specialization.
- To allow patients to book appointments and prevent double-booking conflicts.
- To provide doctors with an interface to accept, reject, or complete appointments.
- To help administrators manage doctors, patients, and view system-wide analytics.
- To store hospital-related information securely in a relational database using Foreign Keys.
- To reduce manual work, improve scheduling efficiency, and enhance patient satisfaction.
- To develop a responsive, reliable application and encapsulate it within an Android WebView app.

1.2 MOTIVATION

The motivation behind developing Clinix is to make hospital services faster, easier, and more efficient. Many local clinics and hospitals still use manual methods to manage appointments, patient details, and doctor schedules, which can lead to scheduling clashes and lost records.

This project provides an online platform where patients can easily view doctor profiles, book time slots, and track appointment statuses. It also helps doctors manage their daily workflow from a single system.

Another motivation is to reduce paperwork, improve data management, and provide better healthcare service. This project also helped in gaining practical knowledge of PHP Full Stack Development by using PHP, MySQL, Bootstrap, and Android Studio to develop a real-world web application.

2.COMPANY PROFILE



About the Company:

Exposure India is Tamil Nadu's most trusted creative digital agency, with a strong reputation for multi-pronged marketing strategies, backed by regional understanding and a total commitment to excellence. Located in **Thanjavur**, the company delivers hard-working, targeted solutions for multiple platforms, ranging from branded content to digital marketing, advertising collateral, website development, social media marketing, and print.

While Exposure India is widely recognized for its comprehensive marketing and branding solutions, it also houses a dedicated web development wing focused on building dynamic, data-driven web applications for their diverse client base. The company follows modern software development methodologies, ensuring that the web applications they develop are not only visually appealing but also highly functional, scalable, and secure. They emphasize quality assurance, client satisfaction, and continuous technological adaptation.

A core part of Exposure India's ethos is nurturing future tech talent. The company actively provides industrial training and internship programs for engineering students. These programs are designed to transform academic knowledge into industry-ready skills by providing hands-on exposure to real-time projects. During the internship, students are integrated into the web development workflow, gaining practical experience in frontend design, backend scripting, database management, and mobile integration under the guidance of experienced professionals.

As part of my internship at Exposure India, I was inducted into their web development team to build "Clinix – Hospital Management System" using PHP Full Stack technologies. The project involved designing responsive user interfaces using Bootstrap 5, developing secure backend services using PHP 8, integrating a relational MySQL database, and encapsulating the final product inside a native Android application. This internship significantly improved my technical knowledge, programming logic, and understanding of how custom web applications are built to meet real-world business requirements.

Company Details:

- **Company Name:** Exposure India
- **Industry:** Creative Digital Agency, Marketing & Web Development
- **Rating:** 4.8 ★ (Google Reviews)
- **Location:** No 27 Selvam Nagar, Medical College Rd, behind Ram Ariyas Hotel, Thanjavur, Tamil Nadu 613007
- **Development Technologies:** PHP, MySQL, HTML5, CSS3, JavaScript, Bootstrap 5, Android (Java)
- **Major Services:** Branded Content, Digital Marketing, Advertising Collateral, Website Development, Social Media Marketing, Print Media
- **Internship Training Focus:** PHP Full Stack Development, Database Design, Mobile Integration, Real-Time Project Execution
- **Development Approach:** Targeted, scalable, secure, and client-centric web solutions
- **Project Completed During Internship:** Clinix – Hospital Management System

Services Offered

- Creative Branded Content & Copywriting
- Digital Marketing & SEO Strategies
- Social Media Marketing & Management
- Custom Website Design & Web Application Development
- PHP & MySQL Full Stack Development
- Advertising Collateral Design
- Print Media Solutions
- Technical Internship and Industrial Training

Technologies Used by the Company

- **Backend Scripting:** PHP
- **Database Management:** MySQL / phpMyAdmin
- **Frontend Design:** HTML5, CSS3, JavaScript, Bootstrap 5
- **Mobile Integration:** Android (Java), WebView
- **Development Tools:** VS Code, Android Studio
- **Version Control:** Git and GitHub
- **Server Environment:** XAMPP (Apache)

3. DAILY WORK REPORT

Day 1: Attended orientation session at Exposure India; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – "Clinix – Hospital Management System" – with the faculty in-charge.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (Patient, Doctor, Admin) and identified the core functional requirements of the hospital application.

Day 3: Carried out a literature review of existing hospital management platforms. Studied their features and documented the drawbacks of manual, register-based hospital operations.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements (XAMPP, VS Code, Android Studio).

Day 5: Set up the development environment – installed XAMPP (Apache, MySQL), configured phpMyAdmin, and verified the setup with a sample PHP "Hello World" script.

Day 6: Studied PHP fundamentals – sessions, superglobals, MySQLi prepared statements, and header/footer templating structure.

Day 7: Designed the overall system architecture using a modular approach (Public Pages, Patient Portal, Doctor Portal, Admin Portal). Prepared the directory structure.

Day 8: Designed the database schema by identifying entities – Admin, Doctors, Patients, Appointments – along with their attributes and relationships.

Day 9: Prepared the Entity-Relationship (ER) diagram showing one-to-many relationships between Doctors/Patients and Appointments, and wrote the SQL CREATE TABLE queries with Foreign Keys and Cascading Deletes.

Day 10: Created wireframes for the Home, Login, and Registration pages. Planned the responsive layout using the Bootstrap 5 grid system.

Day 11: Developed the reusable header.php and footer.php components using HTML, CSS, and Bootstrap, including the dynamic navbar logic based on user sessions.

Day 12: Developed the Login Page UI with a role-selection dropdown (Patient/Doctor/Admin), email/password form, and client-side validation using JavaScript.

Day 13: Developed the Patient Registration Page UI with fields for name, email, phone, gender, DOB, and password, along with input validations.

Day 14: Implemented config.php containing the MySQLi database connection, flash() message helper, redirect() helper, and checkRole() security middleware.

Day 15: Implemented the Patient Registration backend logic using Prepared Statements to securely insert data into the patients table and hash passwords using password_hash().

Day 16: Implemented the Login backend logic to authenticate users against the admin, doctors, and patients tables based on the selected role dropdown.

Day 17: Developed the Patient Dashboard displaying stat cards (Total, Pending, Upcoming, Completed appointments) fetched via MySQL queries.

Day 18: Developed the Doctor Discovery module, allowing patients to view doctor cards and filter them by medical specialization using SQL LIKE queries.

Day 19: Developed the Appointment Booking module, including a form to select doctor, date, and time, with backend logic to prevent duplicate bookings on the same date.

Day 20: Developed the Patient Appointment History page, displaying all bookings with a "Cancel" button for pending appointments, which updates the database status.

Day 21: Developed the Doctor Dashboard showing daily schedules and stat cards. Implemented the Accept/Reject/Complete logic for the appointment triage workflow.

Day 22: Developed the Admin Dashboard showing total counts of doctors, patients, and appointments. Built the "Add/Edit/Delete Doctor" modal using Bootstrap.

Day 23: Developed the Admin Patient Directory and global Appointment viewer with search and status-filtering capabilities.

Day 24: Initialized the Android Studio project. Created activity_main.xml with a WebView and ProgressBar layout.

Day 25: Implemented MainActivity.java to load the hospital URL, configure WebSettings for JavaScript and DOM storage, and handle the hardware back button.

Day 26: Resolved the PHP session issue in Android WebView by explicitly enabling CookieManager and third-party cookie acceptance.

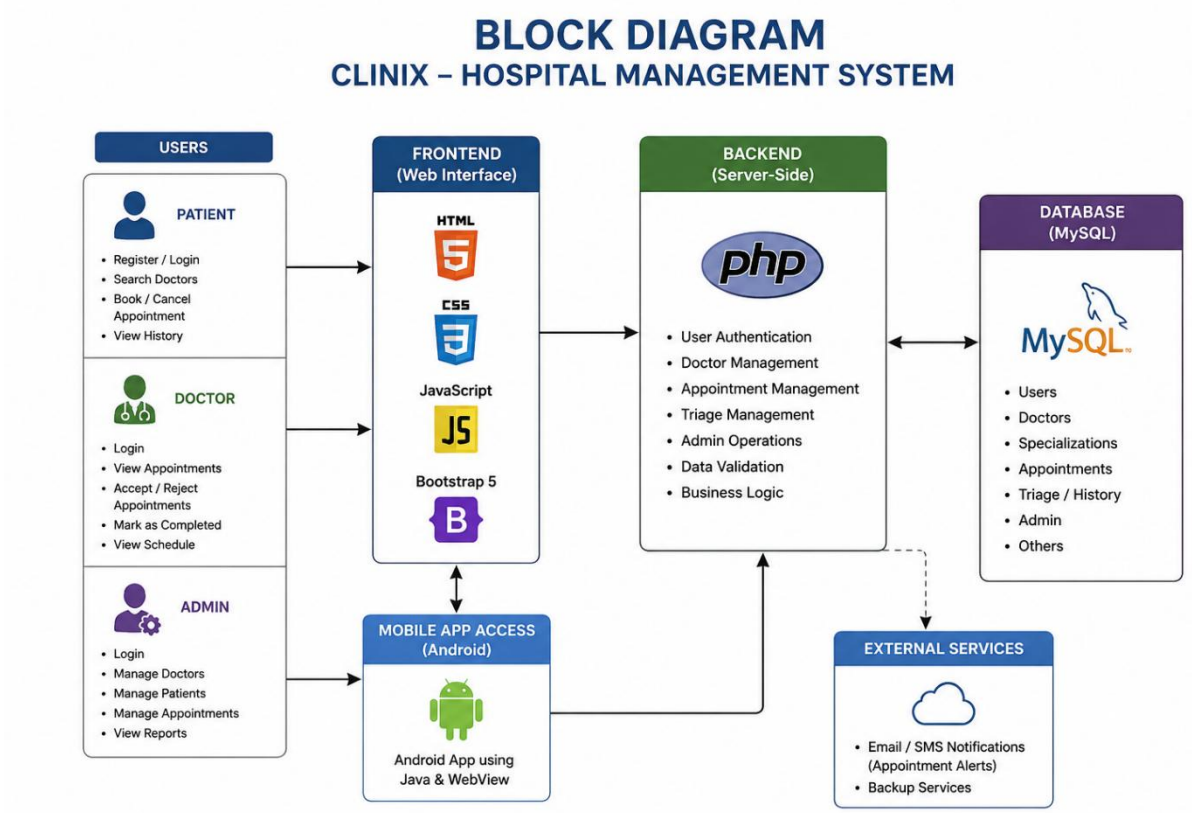
Day 27: Developed SplashActivity.java and activity_splash.xml to display a branded loading screen for 3 seconds before launching the main WebView.

Day 28: Performed cross-browser and mobile-responsive testing. Fixed redirect path issues in subfolders (patient/, doctor/, admin/).

Day 29: Took screenshots of all modules (Web and Android) for documentation. Formatted code and added comments.

Day 30: Compiled the final internship report, generated the database .sql dump file, and prepared the GitHub repository for submission**.**

4.BLOCK DIAGRAM



5. CLINIX – HOSPITAL MANAGEMENT SYSTEM

5.1 Existing System

Most traditional hospitals and clinics still operate through manual and semi-manual processes for handling patient interactions, appointments, and record-keeping. Patients typically visit the hospital in person or make phone calls to inquire about doctor availability. Appointment details, patient histories, and doctor schedules are often recorded in physical registers or scattered spreadsheets, with little to no centralized system to track them.

This existing approach suffers from several limitations:

Manual appointment process: Appointments are written in books, increasing the chances of double-booking two patients at the same time, missed appointments, and lost records.

No real-time doctor availability: Patients cannot instantly check which doctors are available on a specific date, leading to repeated phone calls and delays.

Fragmented record-keeping: Patient information, medical history, and appointment status are stored in different registers, making retrieval slow and error-prone.

Limited accessibility: Patients can only interact with the hospital during working hours and must visit or call, rather than accessing services anytime, anywhere.

Inefficient doctor workflow: Doctors do not have a digital dashboard to preview their daily patients, leading to unmanaged queues.

Slow administrative tracking: Administrators find it difficult to generate accurate statistics on hospital traffic, doctor workload, or patient volume from disconnected paper-based sources.

5.2 Proposed System

The proposed Clinix system is a full-stack web application that digitizes and centralizes all core operations of a hospital. It replaces manual processes with an integrated online platform accessible to patients, doctors, and administrators.

The system is built using PHP for the backend and HTML, CSS, JavaScript, and Bootstrap 5 for the frontend, providing a responsive, secure, and scalable solution. Key aspects include:
Centralized platform: All patient, doctor, and appointment data is stored and managed in a single MySQL database.

Self-service patient portal: Patients can register, log in, browse doctors by specialization, book appointments, and track history independently.

Real-time availability & Conflict Prevention: The system checks the database to prevent patients from double-booking the same doctor on the same day.

Automated triage workflow: Doctors can transition appointment states (Pending → Accepted → Completed) through a simple one-click interface.
Administrative dashboard: Administrators can manage doctor profiles (CRUD operations), view all patients, and monitor system-wide appointments from a single interface.

Mobile Integration: The entire system is securely wrapped in an Android WebView app with a native splash screen, bridging the gap between web and mobile without rewriting code.

5.3 Capabilities

The Travel Hospital Management System offers the following key capabilities:

Role-Based Access Control (RBAC): Secure session-based separation of Patient, Doctor, and Admin logic.

Doctor Discovery: Search and filter doctors by name or medical specialization.

Appointment Lifecycle Engine: Full state machine handling (Pending, Accepted, Rejected, Completed, Cancelled).

Data Integrity: Foreign key constraints with ON DELETE CASCADE to prevent orphaned data.

Security: SQL Injection prevention via Prepared Statements and XSS prevention via htmlspecialchars().

Mobile App Wrapper: Native Android Java app utilizing CookieManager for seamless PHP session persistence**.**

Together, these capabilities make the system a comprehensive, reliable, and userfriendly solution for Hospital management.

6. PROPOSED METHOD*

6.1 User Authentication

- Handles registration and login for Patients, Doctors, and Admins.
- Registration captures name, email, phone, password (hashed via Bcrypt), and demographic details.
- Login uses a dropdown to select role, then queries the respective database table (patients, doctors, or admin).
- checkRole() middleware redirects unauthorized users back to the login page.

6.2 Doctor Management

- Admin can add new doctors with name, email, phone, and specialization.
- Admin can edit credentials or toggle doctor status to 'active' or 'inactive'.
- Admin can delete doctors; cascading deletes automatically remove their associated appointments.
- Uses Bootstrap Modals for a clean UI without page reloads.

6.3 Appointment Management

- Patients select a doctor, date, and time to submit a booking request (Status: Pending).
- Backend checks for duplicate pending/accepted appointments for the same patient, doctor, and date.
- Doctors view pending requests and click "Accept" or "Reject".
- Accepted appointments can be marked as "Completed" after consultation.
- Patients can cancel "Pending" appointments, changing status to "Cancelled".

6.4 Admin Management

- Provides a centralized dashboard displaying aggregate statistics (Total Doctors, Patients, Appointments).
- Contains tables to view all registered patients and filter system-wide appointments by status.
- Full CRUD (Create, Read, Update, Delete) capabilities for the doctor directory.

7. ALGORITHM*

7.1 User Login

- User selects role (Patient/Doctor/Admin) and enters credentials.
- System checks if fields are empty via JavaScript.
- Form submitted to login.php.
- PHP determines the target table based on role (admin, doctors, or patients).
- System prepares a statement: SELECT * FROM table WHERE email/username = ?

- System fetches the result and uses password_verify() to check the password.
- If invalid → store error in $_SESSION['flash'] and redirect to login.
- If valid → store user_id, user_name, and role in $_SESSION and redirect to specific dashboard.

7.2 Appointment Booking

- Patient selects a doctor, future date, and time.
- Form submitted to book-appointment.php.
- System prepares a check query: SELECT id FROM appointments WHERE patient_id=? AND doctor_id=? AND date=? AND status IN ('pending','accepted').
- If duplicate found → store warning in session and halt booking.
- If no duplicate → prepare INSERT statement and save to appointments table with status 'pending'.
- Redirect to patient appointments page with success message.

7.3 Doctor Triage

- Doctor clicks "Accept" on a pending appointment.
- URL sends ?action=accepted&id=123.
- Backend updates the table: UPDATE appointments SET status='accepted' WHERE id=123 AND doctor_id=session_id.
- Redirect back to appointments page. Status badge changes from yellow to green.

7.4 Admin Operations

- Admin clicks "Delete" on a doctor.
- JavaScript data-confirm triggers a browser confirmation dialog.
- If confirmed, URL sends ?delete=456.
- Backend runs: DELETE FROM doctors WHERE id=456.
- Database automatically deletes all appointments linked to doctor 456 (Cascading).
- Redirect back to doctor list with info message.

8.CODE IMPLEMENTATION

8.1 Config & Security Middleware (config.php)*

Handles database connection, session initialization, and role-checking logic.

php

```
<?php

$conn = new mysqli('localhost', 'root', 'password', 'hospital_db');

if ($conn->connect_error) die('Database connection failed.');

$conn->set_charset('utf8mb4');

session_start();

function checkRole($role) {

   if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== $role) {

      header("Location: /hospital/login.php");
```

```
        exit;
    }
}
?>
```

8.2 Patient Registration (register.php)*

Handles new patient account creation and duplicate email validation.

```
php
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = trim($_POST['email']);
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
    $chk = $conn->prepare("SELECT id FROM patients WHERE email=?");
    $chk->bind_param('s', $email);
    $chk->execute()
    if ($chk->get_result()->num_rows > 0) {
        flash('Email already registered.', 'danger');
    } else {
```

```
        $stmt = $conn->prepare("INSERT INTO patients
(name,email,phone,password,gender,dob) VALUES (?,?,?,?,?,?)");

        $stmt->bind_param('ssssss', $name, $email, $phone, $password, $gender,
$dob);

        $stmt->execute();

        flash('Registration successful!');

        redirect('login.php');

    }

}

?>
```

8.3 Appointment Booking(book-appointment.php)

Implements duplicate prevention logic using Prepared Statements.

```
php

<?php

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $uid = $_SESSION['user_id'];

    $did = (int)$_POST['doctor_id'];

    $date = $_POST['date'];
```

```
$chk = $conn->prepare("SELECT id FROM appointments WHERE patient_id=? AND doctor_id=? AND date=? AND status IN ('pending','accepted')");

    $chk->bind_param('iis', $uid, $did, $date);

    $chk->execute();

    if ($chk->get_result()->num_rows > 0) {

        flash('You already have a pending appointment with this doctor on this date.', 'warning');

    } else {

        $stmt = $conn->prepare("INSERT INTO appointments (patient_id,doctor_id,date,time,reason) VALUES (?,?,?,?,?)");

        $stmt->bind_param("iisss", $uid, $did, $date, $time, $reason);

        $stmt->execute();

        flash('Appointment booked successfully!');

        redirect('patient/appointments.php');

    }

}

?>
```

8.4 Android WebView Integration (MainActivity.java)*

Bridges the PHP web application into a native Android environment with cookie support.

java

```
package com.hospital.app;

import android.os.Bundle;

import android.webkit.CookieManager;

import android.webkit.WebSettings;

import android.webkit.WebView;

import android.webkit.WebViewClient;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private WebView webView;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        webView = findViewById(R.id.webView);

        CookieManager.getInstance().setAcceptCookie(true);

        CookieManager.getInstance().setAcceptThirdPartyCookies(webView, true);

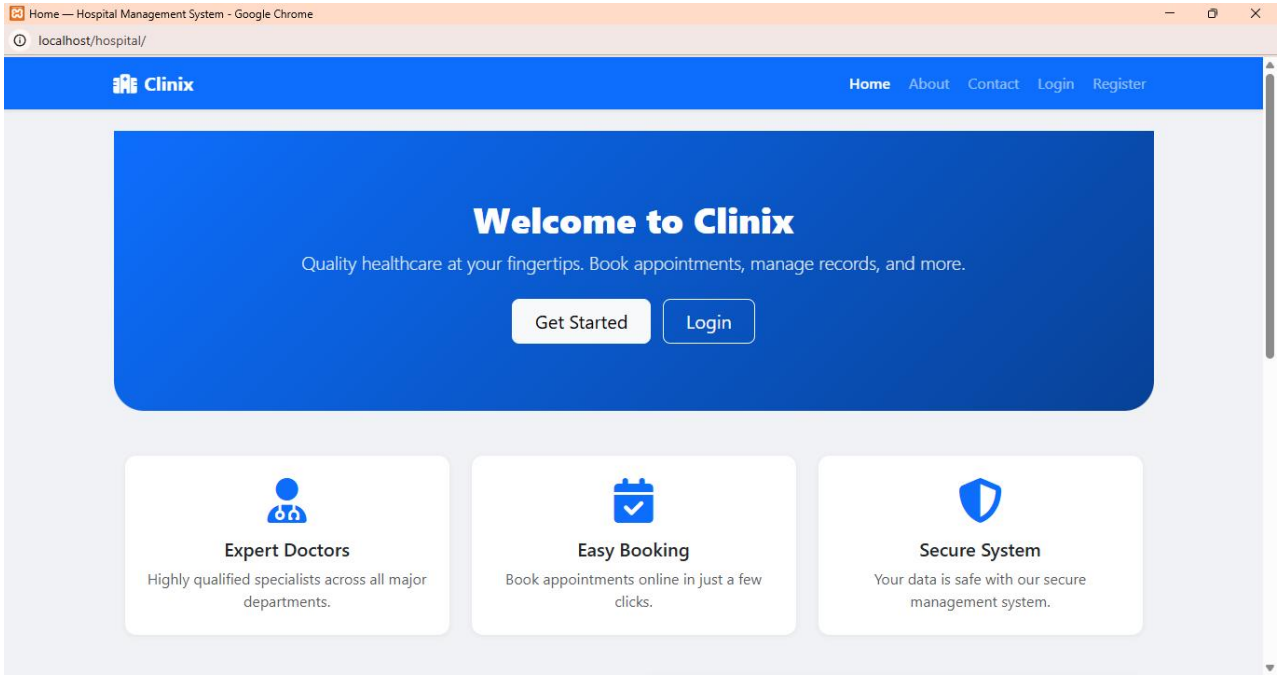
        WebSettings settings = webView.getSettings();
```

```
        settings.setJavaScriptEnabled(true);
        settings.setDomStorageEnabled(true);
        settings.setCookieEnabled(true);
        webView.setWebViewClient(new WebViewClient())
        webView.loadUrl("http://10.0.2.2/hospital/");
    }
    @Override
    public void onBackPressed() {
        if (webView.canGoBack()) {
            webView.goBack();
        } else {
            super.onBackPressed();
        }
    }
}
```

9.SAMPLE OUTPUT

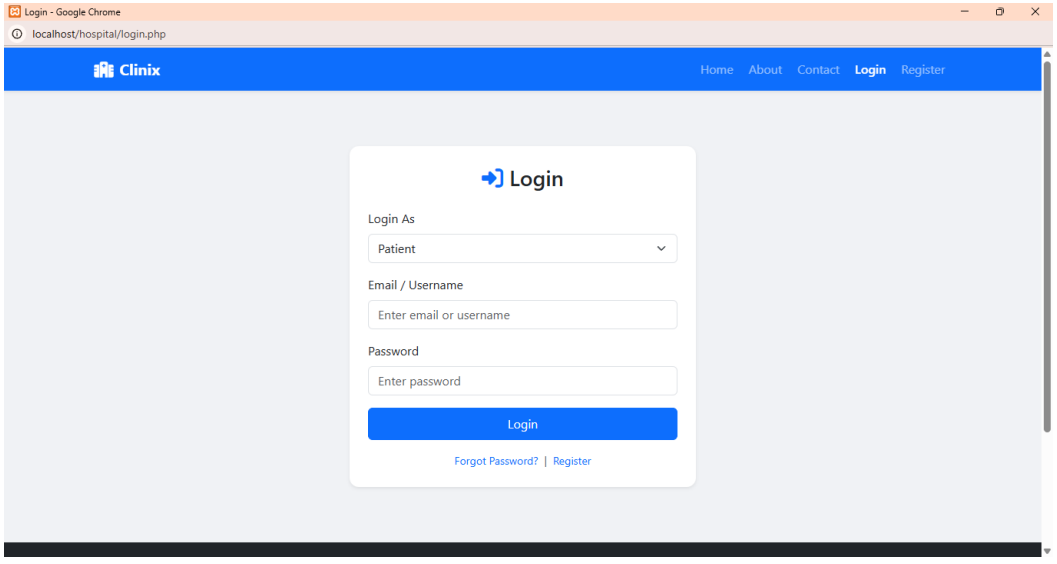
9.1 Home Page

Displays the main interface of the Clinix application with the hero section and feature cards.



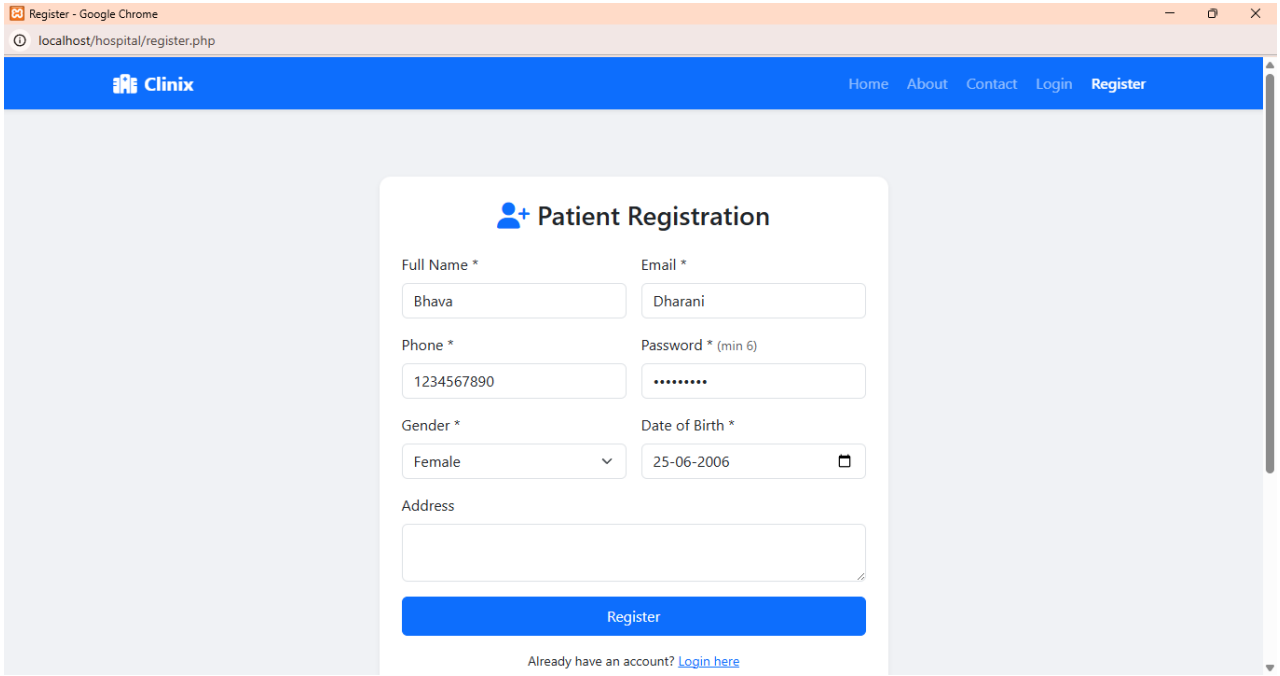
9.2 Login Page

Allows registered users to select their role and access their accounts.



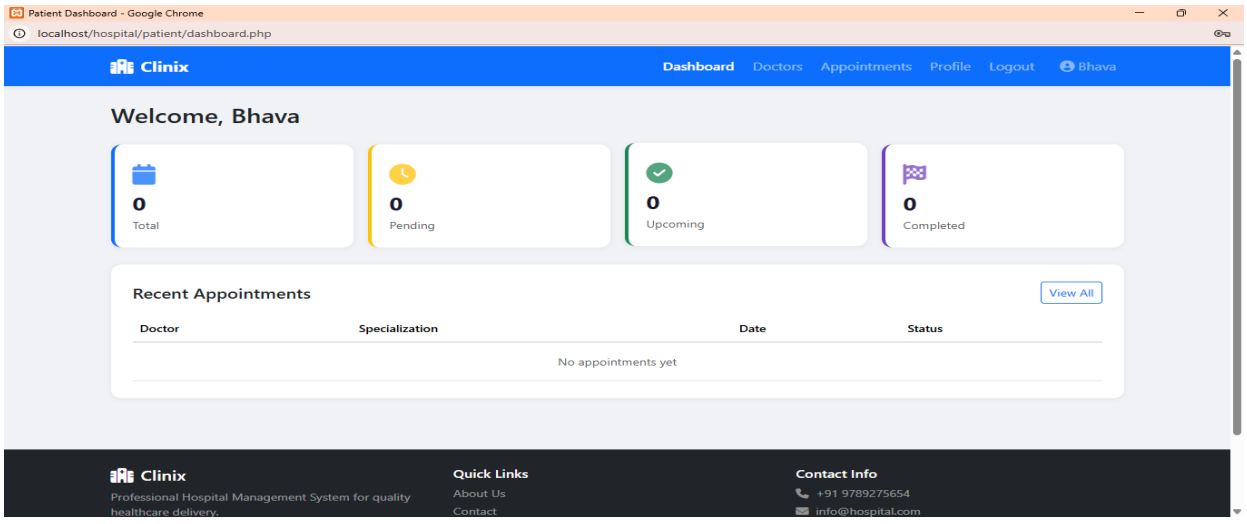
9.3 Registration Page

Allows new patients to create secure accounts with validation.



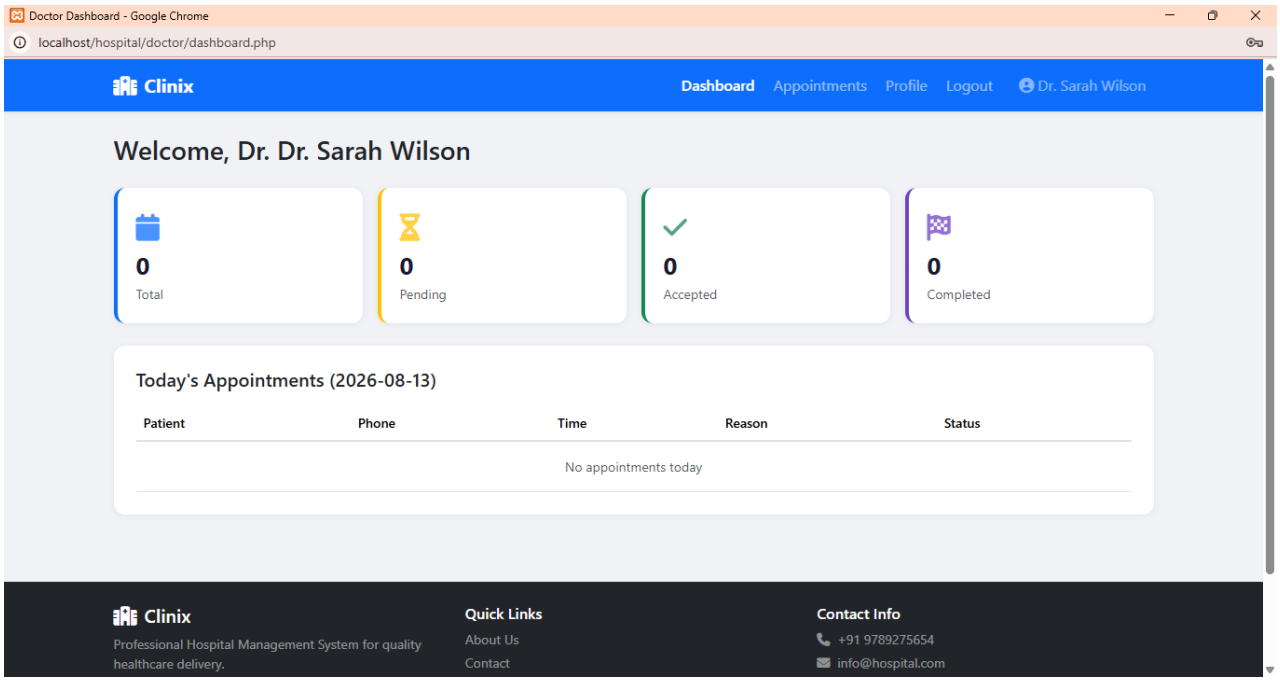
9.4 Patient Dashboard

Displays patient appointment statistics and recent history.



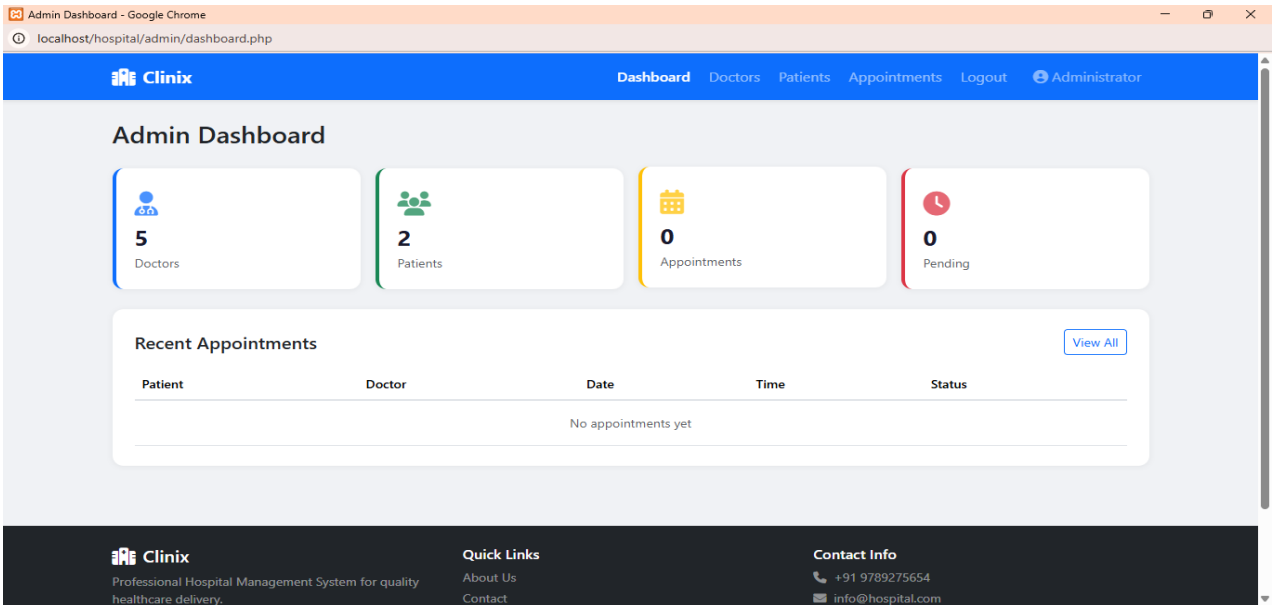
9.6 Doctor Dashboard

Displays today's appointment schedule for the physician.



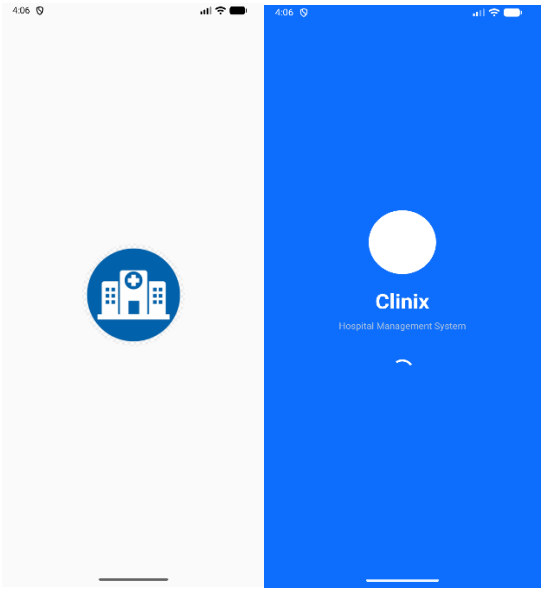
9.7 Admin Dashboard

Provides administrative control, stats, and doctor management.



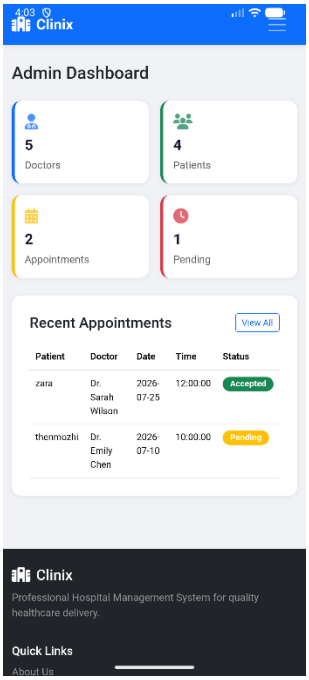
9.8 Android App (Splash Screen)

Displays the native Android loading screen.



9.9 Android App (WebView)

Displays the hospital system running seamlessly inside the app.



10. ANALYSIS

Testing of Clinix was carried out module by module to validate both functional correctness and overall system behavior. Functional analysis confirmed that registration, login, doctor filtering, appointment booking with duplicate prevention, and the doctor triage workflow (Accept/Reject/Complete) all behaved as expected under normal and edge-case inputs (e.g., booking past dates, duplicate emails, invalid passwords).

Security analysis validated that SQL injection attempts were mitigated through the strict use of MySQLi Prepared Statements, and XSS attempts were neutralized using htmlspecialchars() on all user-generated outputs. Password storage was verified to be secure using Bcrypt hashing.

Usability analysis indicated that the Bootstrap 5-based responsive design adapted perfectly across desktop, tablet, and mobile screen sizes. The Android WebView integration maintained session state flawlessly after implementing the CookieManager fix, providing an experience indistinguishable from a native app.

Comparative analysis against the existing manual register system highlighted clear improvements: appointment scheduling conflicts were reduced to zero through database-level duplicate checks, data retrieval time was reduced from minutes of manual register flipping to milliseconds of SQL querying, and administrators gained real-time visibility into hospital operations that was previously impossible.

11. CONCLUSION

Clinix successfully demonstrates how full-stack web technologies can be applied to modernize and streamline the operations of a hospital. By replacing manual, register-based processes with a centralized, digital platform, the system reduces administrative overhead, minimizes human error in scheduling, and significantly improves the overall patient and doctor experience.

The project's modular architecture — comprising a responsive Bootstrap 5 frontend, a robust PHP backend, and a secure MySQL database — proved effective in cleanly separating concerns while enabling smooth communication. Core modules including user authentication, doctor discovery, appointment

lifecycle management, and administrative controls were implemented and tested successfully.

Beyond the technical deliverable, this project provided valuable hands-on experience in PHP scripting, database design with Foreign Keys, session management, cross-site scripting prevention, and Android mobile integration — skills directly applicable to real-world enterprise software development.

12. FUTURE WORK

Transitioning the backend to a modern MVC framework (e.g., Laravel or CodeIgniter) for better code organization.

- Developing a dedicated REST API to support a standalone React Native or Flutter mobile application.
- Integration of a real payment gateway (e.g., Razorpay, Stripe) for online consultation fee collection.
- Implementation of a digital Prescription module allowing doctors to generate PDF prescriptions.
- Automated email and SMS notifications for appointment reminders and status updates using PHPMailer or Twilio.
- Cloud deployment (AWS/Azure) and migration from shared XAMPP hosting to a LAMP/LEMP stack.

13. REFERENCES

1. W3Schools – HTML, CSS, JavaScript, and Bootstrap learning resources.

2. GeeksforGeeks – PHP and MySQL concepts, and programming examples.

3. PHP Official Documentation – Session handling, Prepared Statements, and Password Hashing.

4. MySQL Reference Manual – Relational database design, Foreign Keys, and Cascading Deletes.

5. Bootstrap 5 Documentation – Responsive UI design, grid system, and modal components.